Лабораторная работа №4. Зависимости и контейнер: Trivy и SBOM

Тема: анализ состава программного обеспечения (SCA), перечень компонентов (SBOM), проверка Dockerfile и образа. **Время:** 2 академических часа. **Результат:** отчёты Trivy «до» и «после», SBOM, обновлённые зависимости, контейнер без прав root, закрытый порт базы данных.

Перед началом: выполнена ЛР3, установлен и запущен Docker.

---

1. Цель работы

Научиться проверять не только свой код, но и чужой — библиотеки и базовые образы, из которых собрано приложение, — и принимать по находкам обоснованные решения.

После работы вы сможете:

- объяснить, что такое прямая и транзитивная зависимость, CVE и CVSS;
- запускать Trivy по каталогу, по конфигурации и по образу;
- сформировать SBOM и объяснить, зачем он нужен;
- отличать «уязвимость найдена» от «уязвимость опасна для нас»;
- устранять типовые ошибки Dockerfile и Compose.

2. Теория

2.1. Чужой код в вашем проекте

Современное приложение на 80–90 % состоит из кода, который вы не писали: библиотек, фреймворков, базового образа операционной системы.

- **Прямая зависимость** — библиотека, которую вы сами указали в `requirements.txt`. У нас это Flask.
- **Транзитивная зависимость** — библиотека, которую потянула за собой прямая. Flask приводит с собой Werkzeug, Jinja2 и другие.

Уязвимость в любой из них — уязвимость вашего приложения. При этом вы за неё отвечаете, хотя не писали ни строки этого кода.

2.2. SCA

SCA (Software Composition Analysis) — анализ состава ПО. Инструмент:

1. находит файлы с перечнем зависимостей (`requirements.txt`, `package-lock.json` и т. п.) или установленные пакеты внутри образа;
2. определяет название и версию каждого компонента;

3. сверяет их с базами известных уязвимостей.

SCA не читает код библиотек и не ищет в них новые ошибки. Он отвечает на вопрос: «есть ли среди наших компонентов версии, про которые **уже известно**, что они уязвимы?»

2.3. CVE, CVSS и риск

- **CVE** (Common Vulnerabilities and Exposures) — уникальный идентификатор известной уязвимости, например `CVE-2019-19844`.
- **CVSS** — числовая оценка тяжести от 0 до 10. Trivy показывает её словами: LOW, MEDIUM, HIGH, CRITICAL.
- **Fixed Version** — версия, в которой уязвимость исправлена.

CVSS описывает уязвимость «в вакууме». Риск для вашего проекта зависит от вопросов, на которые сканер не отвечает:

- используется ли этот компонент вообще? попадает ли он в то, что запускается?
- вызывается ли в вашем коде уязвимая функция?
- доступна ли она нарушителю извне?

Поэтому результат SCA — не приговор, а список вопросов для разбора. Возможные решения: обновить, удалить компонент, применить компенсирующую меру, принять риск на ограниченный срок с обоснованием.

2.4. SBOM

SBOM (Software Bill of Materials) — перечень компонентов, из которых состоит программа: названия, версии, типы. Аналог состава на упаковке продукта.

Зачем он нужен, если есть сканер? Представьте, что завтра объявят о критической уязвимости в популярной библиотеке. Вопрос «в каких наших выпусках она есть?» без SBOM требует поднять и пересканировать каждую версию каждого продукта. С SBOM это поиск по файлам.

Распространённые форматы — **CycloneDX** и SPDX. Мы используем CycloneDX.

SBOM сам уязвимости не находит. Это опись, а не проверка.

2.5. Безопасность контейнера

Контейнер изолирует приложение, но изоляция не абсолютна. Типовые ошибки:

Ошибка	Чем опасна	Как исправить
Процесс работает от `root`	при взломе приложения нарушитель получает максимальные права внутри контейнера, а при ошибке изоляции — и на сервере	инструкция `USER` с обычным пользователем
Наружу открыты	база данных доступна из сети в обход	публиковать только

Ошибка	Чем опасна	Как исправить
лишние порты	приложения	порт приложения
Версия образа не зафиксирована (`latest`)	сегодня и завтра собираются разные образы; результат невоспроизводим	указывать версию
В образ попадают лишние файлы	утечка секретов, лишние уязвимости	`.dockerignore`, копировать только нужное

2.6. Почему версия сканера тоже фиксируется

Сканер безопасности — такая же зависимость, как библиотека: он получает доступ к вашему коду, а в CI — и к секретам сборки. Известны случаи, когда злоумышленники подменяли опубликованные версии популярных инструментов. Поэтому в практикуме у каждого инструмента указана конкретная версия, а не `latest`. То же правило вы примените в ЛР5 к конвейеру.

3. Ход работы

Все команды выполняются в каталоге `~/devsecops-lab` с активированным окружением. В Windows (Git Bash) сначала выполните `export MSYS_NO_PATHCONV=1`.

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
```

Шаг 1. Просканировать зависимости

```
docker run --rm aquasec/trivy:0.76.2 --version
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --scanners vuln /src
```

Что означают параметры:

- `fs` — проверять каталог с файлами;
- `--scanners vuln` — искать только уязвимости в зависимостях;
- `-v trivy-cache:/root/.cache/trivy` — хранить базу уязвимостей между запусками, чтобы не скачивать её каждый раз.

Первый запуск скачивает базу уязвимостей и занимает 1–2 минуты.

Ожидаемый результат: сводная таблица с двумя строками, затем подробные таблицы по каждому файлу.

Файл	Уязвимостей в контрольном запуске
`legacy-component/requirements.txt`	44
`requirements.txt`	1

Ваши числа будут **больше**: база уязвимостей пополняется каждый день. Это нормально и само по себе важный вывод: проект, в котором ничего не меняли, со временем становится уязвимее.

Сохраните отчёт в файл:

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --scanners vuln --format json --output /src/reports/trivy-fs-before.json
 /src
```

В отчёт: версия Trivy, дата запуска, сводная таблица.

Шаг 2. Разобрать находки

Покажите только самые тяжёлые:

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --scanners vuln --severity HIGH,CRITICAL /src
```

Выберите пять уязвимостей: по одной для Django, requests, urllib3 и Jinja2 из `legacy-component` и одну из основного `requirements.txt` (её видно только без фильтра `--severity`). Для каждой откройте ссылку из колонки Title и заполните таблицу. Первая строка — образец.

Библиотека	Версия	CVE	Тяжесть	Исправлено в	Суть уязвимости своими словами
requests	2.19.0	CVE-2018-18074	HIGH	2.20.0	при перенаправлении с HTTPS на HTTP библиотека отправляет заголовок с паролем в открытом виде
Django					
urllib3					
Jinja2					
Flask					

В отчёт: таблица пяти уязвимостей.

Шаг 3. Сформировать SBOM

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --format cyclonedx --output /src/reports/sbom-before.cdx.json /src
```

Откройте `reports/sbom-before.cdx.json` в редакторе и найдите:

- поле `bomFormat` и версию спецификации `specVersion`;
- массив `components` — сколько в нём элементов и какого они типа (`type`).

Ожидаемый результат: `CycloneDX`, 9 компонентов — 7 библиотек (`library`) и 2 файла с зависимостями (`application`).

Ответьте письменно: в SBOM нет библиотеки Werkzeug, хотя приложение её использует (проверьте командой `pip list`). Почему? Что нужно сканировать, чтобы увидеть полный состав? Вернитесь к этому вопросу после шага 7.

В отчёт: число компонентов, ответ на вопрос.

Шаг 4. Оценить, что из найденного опасно на самом деле

44 уязвимости в `legacy-component` выглядят страшно. Проверим три факта.

Факт 1. Импортирует ли наш код эти библиотеки?

```
grep -rnE "^(import|from) +(django|requests|urllib3)" backend/
```

Ожидаемый результат: пусто.

Факт 2. Устанавливаются ли они при сборке образа? Откройте `Dockerfile` и найдите, какой файл передаётся в `pip install`. Затем откройте `.dockerignore`.

Факт 3. Есть ли они в собранном образе?

```
docker build -t devsecops-lab:local .
docker run --rm devsecops-lab:local pip list
```

Ожидаемый результат: в списке есть Flask, Werkzeug, Jinja2 версии 3, psycopg, gunicorn; нет Django, requests и urllib3.

Сделайте вывод и запишите его по образцу:

> Компонент `legacy-component` обнаружен в репозитории. В работающее приложение не входит: (доказательства 1–3). Риск для работающего сервиса — …. Риск для проекта — … (подсказка: что будет, если кто-то решит этот файл установить? как повлияет постоянный «красный» отчёт на внимание к новым находкам?). Решение: ….

В отчёт: три проверки и вывод.

Шаг 5. Устранить находки в зависимостях

Два разных решения для двух разных ситуаций.

Неиспользуемый компонент — удалить. Лучший способ избавиться от уязвимостей в ненужном коде — убрать код:

```
git rm -r legacy-component
```

Используемую библиотеку — обновить. В `requirements.txt` замените `Flask==3.0.3` на `Flask==3.1.3` и переустановите зависимости:

```
python -m pip install -r requirements.txt
pytest -q
```

Ожидаемый результат: `7 passed`. Тесты после обновления обязательны: новая версия библиотеки может изменить поведение.

Повторите сканирование так, как оно будет работать в CI, — с порогом:

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --scanners vuln --severity HIGH,CRITICAL --exit-code 1 /src
echo "Код завершения: $?"
```

Параметр `--exit-code 1` заставляет Trivy вернуть ошибку, если найдены уязвимости указанной тяжести.

Ожидаемый результат: `requirements.txt | pip | 0`, код завершения 0.

Если Trivy всё же нашёл уязвимость в `requirements.txt` — значит, после выхода этой методички опубликовали новую. Поступите как специалист: посмотрите Fixed Version, обновите библиотеку, прогоните тесты. Опишите это в отчёте.

В отчёт: вывод `pytest`, вывод Trivy с кодом завершения.

Шаг 6. Проверить Dockerfile и Compose

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 config /src
```

Ожидаемый результат: одна находка в `Dockerfile` — **DS-0002 (HIGH)**: не задана инструкция `USER`, контейнер работает от `root`.

Убедитесь в этом на деле — образ вы уже собрали на шаге 4:

```
docker run --rm devsecops-lab:local id
```

Ожидаемый результат: `uid=0(root)`.

Теперь изучите `compose.yaml` самостоятельно — Trivy этот файл не проверяет. Найдите раздел `ports` у сервиса `db`. Что он означает? Нужно ли приложению, чтобы порт базы был открыт на вашем компьютере, если `web` и `db` общаются внутри сети Compose?

Исправление 1. В `Dockerfile` перед строкой `ENV PYTHONPATH=/app` добавьте:

```
RUN useradd --create-home labuser && chown -R labuser:labuser /app
USER labuser
```

Первая строка создаёт обычного пользователя и передаёт ему каталог приложения, вторая — переключает на него все последующие команды и запуск контейнера.

Исправление 2. В `compose.yaml` у сервиса `db` удалите две строки:

```
    ports:
      - "5432:5432"
```

Раздел `ports` у сервиса `web` не трогайте.

Проверьте:

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 config --exit-code 1 /src
docker compose up -d --build --wait
```

```
docker compose exec web id
curl http://127.0.0.1:5000/health
```

Ожидаемый результат:

- Trivy: `Dockerfile | dockerfile | 0`;
- `uid=1000(labuser)`;
- `{"database":"postgresql","status":"ok"}` — приложение работает с базой, хотя её порт наружу больше не открыт.

Откройте http://127.0.0.1:5000 и убедитесь, что вход `student / student123` работает.

Если в YAML-файле после правки ошибка — проверьте отступы: в YAML они значимы, и табуляция не допускается.

В отчёт: находка DS-0002, вывод `id` до и после, ваш анализ порта 5432.

Шаг 7. Просканировать образ

До сих пор Trivy смотрел на файлы проекта. Теперь проверим то, что реально запускается, — образ вместе с операционной системой и всеми установленными пакетами.

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/roo
t/.cache/trivy aquasec/trivy:0.76.2 image --severity HIGH,CRITICAL devsecops-lab
-web
```

Параметр `-v /var/run/docker.sock:…` даёт Trivy доступ к образам на вашем компьютере. Имя образа `devsecops-lab-web` присвоил Compose: имя каталога плюс имя сервиса. Если ваш каталог называется иначе, посмотрите имя командой `docker images`.

Ожидаемый результат: десятки находок в пакетах операционной системы Debian и, возможно, несколько в служебных пакетах Python. В контрольном запуске — 49 HIGH в Debian, из них только у двух есть исправление.

Откуда они взялись, если в `requirements.txt` чисто? Из базового образа `python:3.11-slim`: в нём целая операционная система. Посмотрите на колонки Status и Fixed Version: у большинства находок исправления ещё нет (`affected`), и обновить вам нечего.

Покажите только те, для которых исправление существует:

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/roo
t/.cache/trivy aquasec/trivy:0.76.2 image --severity HIGH,CRITICAL --ignore-unfi
xed devsecops-lab-web
```

Исправлять их в этой работе не нужно. Нужно ответить письменно:

1. Сколько находок HIGH и CRITICAL всего и сколько из них имеют исправление?
2. Почему нельзя просто поставить условие «образ без единой HIGH-уязвимости»?
3. Какие есть способы уменьшить их число? (Подсказка: регулярная пересборка, более свежий или более компактный базовый образ.)

4. Теперь вернитесь к вопросу из шага 3: где Trivy увидел Werkzeug и Jinja2?

Остановите приложение:

```
docker compose down -v
```

В отчёт: числа находок и ответы на четыре вопроса.

Шаг 8. SBOM «после» и правила приёмки

```
docker run --rm -v "$PWD:/src" -v trivy-cache:/root/.cache/trivy aquasec/trivy:0.
76.2 fs --format cyclonedx --output /src/reports/sbom-after.cdx.json /src
```

Ожидаемый результат: в новом SBOM 4 компонента вместо 9.

Сформулируйте **правила приёмки (quality gate)** — условия, при которых изменение не принимается. Они понадобятся в ЛР5. Заполните таблицу:

Сигнал	Действие	Кто решает	Срок
Новая HIGH/CRITICAL в зависимостях приложения, исправление есть	блокировать, обновить, прогнать тесты	разработчик	до слияния
Новая HIGH/CRITICAL, исправления нет			
Уязвимость в базовом образе без исправления			
Находка в Dockerfile (например, DS-0002)			
Сканер не смог скачать базу уязвимостей			

Про последнюю строку подумайте особенно: если сканер не отработал, проверка **пройдена** или **не выполнена**?

В отчёт: таблица правил приёмки.

Шаг 9. Зафиксировать результат

```
git status
git add -A
git commit -m "lab4: update Flask, drop unused legacy manifest, non-root contain
er, close DB port"
git log --oneline
```

Ожидаемый результат: хук `Detect hardcoded secrets` показал `Passed`, в истории пять коммитов.

4. Требования к отчёту

1. Версия Trivy, дата запуска, сводная таблица первого сканирования.
2. Таблица пяти уязвимостей с описанием сути.

3. Сведения о SBOM «до» и «после», ответ на вопрос о полноте.
4. Анализ `legacy-component`: три проверки и вывод.
5. Результаты после удаления и обновления: `pytest`, Trivy с кодом завершения.
6. DS-0002, анализ порта базы данных, изменения в `Dockerfile` и `compose.yaml`, вывод `id` до и после.
7. Результаты сканирования образа и ответы на вопросы.
8. Таблица правил приёмки.
9. Вывод.

5. Контрольные вопросы

1. Чем SCA отличается от SAST? Может ли один заменить другой?
2. Что такое транзитивная зависимость и почему её уязвимости — ваша проблема?
3. Почему оценка CVSS не равна риску для конкретного проекта?
4. Чем SBOM отличается от отчёта об уязвимостях? Зачем хранить SBOM каждого выпуска?
5. Почему число находок в одном и том же проекте растёт со временем?
6. Чем опасен запуск процесса в контейнере от `root`?
7. Что делать, если для уязвимости нет исправленной версии?
8. Почему версию сканера нужно фиксировать так же, как версию библиотеки?

6. Критерии оценивания

Критерий	Баллы
Сканирование зависимостей и таблица уязвимостей	20
SBOM и его разбор	10
Анализ применимости (`legacy-component`)	20
Устранение находок в зависимостях, тесты	15
Dockerfile и Compose: находки, исправления, проверка	20
Сканирование образа, правила приёмки, выводы	15
Итого	**100**

7. Источники

- ГОСТ Р 56939-2024. Разработка безопасного программного обеспечения. Общие требования.
- Trivy: https://trivy.dev/
- CycloneDX: https://cyclonedx.org/
- OWASP Docker Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- База уязвимостей NVD: https://nvd.nist.gov/